Exercise 2 : E commerce Platform Search Function

Big O Notation:

Big O Notation is a mathematical tool used in computer science to describe the performance or complexity of an algorithm. It details how the execution time or space requirements grow as the input data size (n) increases.

Best-case scenario:

The algorithm performs the minimum number of operations. For example, finding the target element on the very first attempt.

Average-case scenario:

The expected performance when the target element is located randomly within the dataset. For example, the target element is somewhere in between of the array or dataset.

Worst-case scenario:

The absolute maximum number of operations required. For example, the target element is at the very end of the dataset or does not exist at all.

1. Product.java

```
public class Product {
    private int id;
    private String name;

    public Product(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

2. SearchStrategy.java

```
public interface SearchStrategy {
    Product search(Product[] products, int id);
}
```

3. LinearSearch.java

```
public class LinearSearch implements SearchStrategy {

    @Override
    public Product search(Product[] products, int id) {

        for (Product product : products) {
            if (product.getId() == id) {
                return product;
            }
        }

        return null;
    }
}
```

4.BinarySearch.java

```
public class BinarySearch implements SearchStrategy {

    @Override
    public Product search(Product[] products, int id) {

        int left = 0;
        int right = products.length - 1;

        while (left <= right) {

            int mid = (left + right) / 2;

            if (products[mid].getId() == id)
                return products[mid];

            if (products[mid].getId() < id)
                left = mid + 1;
            else
```

```
            right = mid - 1;
        }

        return null;
    }
}
```

5. SearchContext.java

```
public class SearchContext {

    private SearchStrategy strategy;

    public SearchContext(SearchStrategy strategy) {
        this.strategy = strategy;
    }

    public Product executeSearch(Product[] products, int id) {
        return strategy.search(products, id);
    }
}
```

6.StrategyPatternTest.java

```
import java.util.Arrays;
import java.util.Comparator;

public class StrategyPatternTest {

    public static void main(String[] args) {

        Product[] products = {
                new Product(103, "Keyboard"),
                new Product(101, "Laptop"),
                new Product(104, "Monitor"),
                new Product(102, "Mouse")
        };

        // Linear Search
        SearchContext context = new SearchContext(new LinearSearch());

        Product result = context.executeSearch(products, 102);

        if (result != null)
            System.out.println("Linear Search: " + result.getName());
```

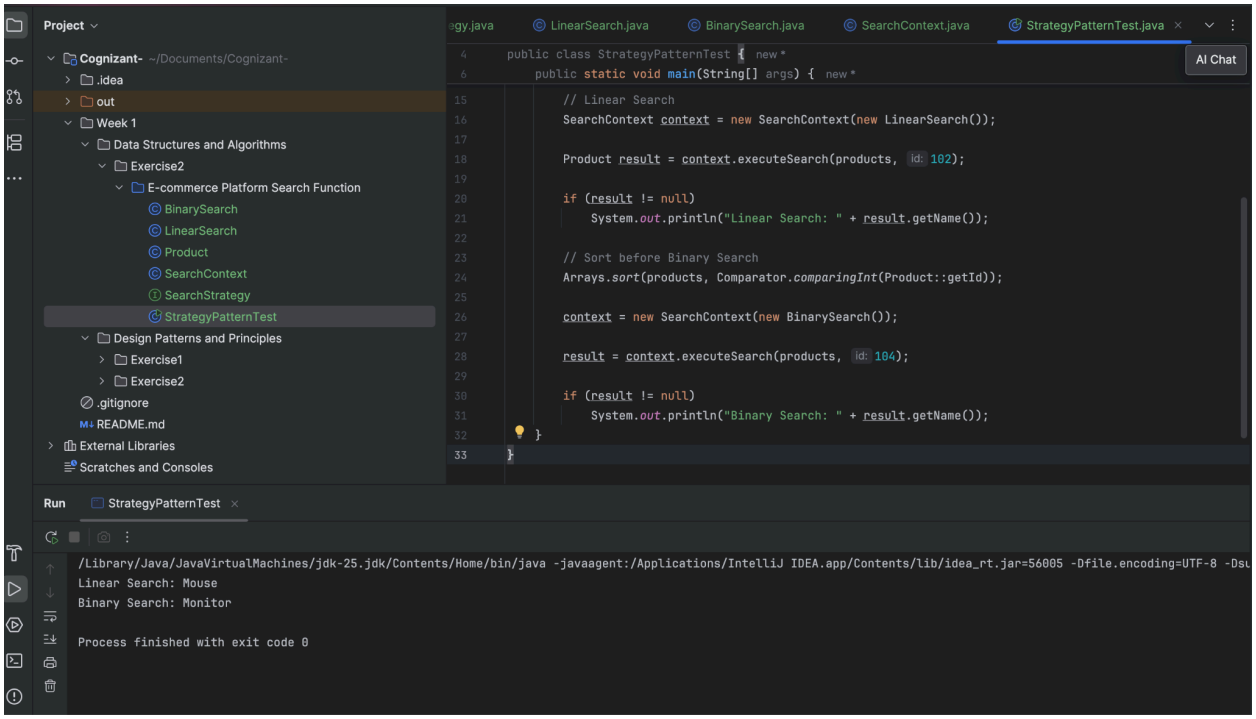
```
        // Sort before Binary Search
        Arrays.sort(products, Comparator.comparingInt(Product::getId));

        context = new SearchContext(new BinarySearch());

        result = context.executeSearch(products, 104);

        if (result != null)
            System.out.println("Binary Search: " + result.getName());
    }
}
```

Output:



Time Complexity Comparison:

Algorithm	Best-Case	Average-Case	Worst-Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)